

Architecture platforms for future vehicles: a comparison of ROS2 and Adaptive AUTOSAR

Jacqueline Henle

Embedded Systems and Sensors Engineering (ESS)
FZI Research Center for Information Technology
Berlin, Germany
henle@fzi.de

Martin Stoffel

Middleware Engineering Autonomous Truck
Daimler Truck AG
Stuttgart, Germany
martin.stoffel@daimlertruck.com

Marc Schindewolf, Ann-Therese Nägele, Eric Sax
Institute for Information Processing Technologies (ITIV)
Karlsruhe Institute of Technology
Karlsruhe, Germany
marc.schindewolf@kit.edu, ann-therese.naegele@kit.edu,
eric.sax@kit.edu

Abstract—Autonomous Vehicles (AVs) are supposed to be continuously updated, besides that they combine many diverse applications by various developers. Accordingly, they require flexible and dynamic software (SW) architectures. With communication paradigms changing from signal-based to service-oriented architectures (SOAs), long-established standards seem no longer sufficient. The AUTomotive Open System ARchitecture (AUTOSAR) Platform came up with the Adaptive standard to meet future automobile requirements. Alongside, while requirements of AVs and robotic systems are similar to a certain extent, the Robot Operating System (ROS) gets growing attention in the automotive environment. With the introduction of ROS2 and ROS2-based commercial solutions, the Adaptive AUTOSAR Platform seems not to be the only appropriate standard for developing high-performing SW architectures. This paper provides an overall picture of Adaptive AUTOSAR and ROS2 and evaluates ROS2 and its suitability in an automotive context based on the Adaptive AUTOSAR architecture. This research analyzes the extent to which ROS2 fulfills the functionality provided by Adaptive AUTOSAR.

Index Terms—ROS2, Adaptive AUTOSAR, Automotive, Service-oriented Architecture

I. INTRODUCTION

In recent years the automation, digitization and growing connection of various systems have challenged several industries. In the automotive sector, semi-automated vehicles are under development or already on the street. Real-time capabilities as well as flexibility, safety and security need to be guaranteed by such cars. These trends result in a transformation of automotive electric and electronic architectures (E/E architectures) [1].

Today a modern vehicle architecture connects more than 100 Electronic Control Units (ECUs) [2]. Applications usually run on a predefined ECU, communicating based on a preset signal-based Communication Matrix (C-Matrix) [3]. By using bus networks such as Controller Area Network (CAN) or FlexRay, values of sensors are collected and transmitted to a receiving ECU according to the C-Matrix.

The requirements arising with Advanced Driver Assistance Systems (ADAS) functions and AVs resulted in static E/E-architectures and signal-based communication to reach efficiency limits [4]. On the technical side, more powerful bus networks, such as Ethernet, are introduced to provide more bandwidth and satisfy further requirements of future cars [5]. Besides the shift of requirements, these advances pave the way for more dynamic communication in the form of a SOA. The communication of various components is realized by providing services in a distributed system. Service consuming entities subscribe dynamically to these services and enable system interaction. While the static C-Matrix as the base of signal-based communication is hardly flexible, service-oriented communication paths are dynamically established during runtime [6].

In contrast to the ECU-focused development, the SW application deployment in a SOA is no longer statically coupled with the underlying Hardware (HW) [7]. To enable this kind of dynamic SW architecture, interoperability between the service sending and receiving entity is essential.

Standardization is important in the signal-based communication pattern to realize cost-efficient and reliable deeply embedded systems. The SOA requires standardization and harmonization of higher architecture abstraction layers and interfaces. Besides, the middleware layer in the role of a coordinator handling the service-based communication becomes a key enabler of high-performing SOAs.

In the automotive industry, AUTOSAR is an international consortium composed of automotive Original Equipment Manufacturers (OEMs), suppliers and other industry members. They released the first reference architecture AUTOSAR Classic in 2003. Its focus does not lie on dynamic applications independent of the underlying HW. But yet, with the growing importance of SOA and AVs, the release of a second standard, Adaptive AUTOSAR in 2018, addresses these requirements [5].

However, the automotive sector is not the first to face a paradigm change toward dynamic and flexible communication patterns. In the environment of Cyber-Physical Systems and Robotics, a similar evolution took place. In this context, the widely used ROS, an open-source set of libraries and tools for SW architecture development, did not satisfy the real-time and safety needs as well as the cross-platform interoperability required [8]. The successor ROS2 addresses these shortcomings, inter alia, and enables service-oriented communication. ROS2 experiences broad usage in scientific environments to realize SW encapsulation and communication, also in the context of automotive E/E architectures [9]. Meanwhile, there are even solutions certified to satisfy the functional safety standard of electrical and/or electronic systems (ISO26262) and the necessary Automotive Safety Integrity Level (ASIL) [10].

Many questions regarding SOAs in the context of ADASs functions and highly connected cars are still to be solved. In this context, our work compares ROS2 and Adaptive AUTOSAR, examining the adequacy for automotive E/E architectures (visualized in Fig. 1). Based on the architecture standardized by Adaptive AUTOSAR, the similarities and differences of ROS2 are examined.

This comparison aims to identify their assets and drawbacks, the rationale behind using a standard and their suitability for future-ready E/E architectures of modern vehicles.

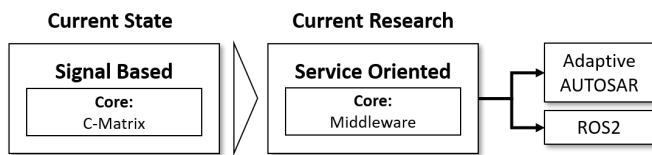


Fig. 1: From Signal to Service

II. RELATED WORK

To analyze the SW architecture suitable and the attached design drivers for developing autonomous driving functions, Berger and Dukaczewski [11] conducted a case study examining ROS and Classic AUTOSAR realizations. Distinguished therefrom, this paper focuses on the subsequent platforms Adaptive AUTOSAR and ROS2.

Trachtenherz [12] describes the logical architecture as the system structure from a logical point of view, which includes required functionalities provided by the system. According to Baun [13], logical topology refers to the data flow between components. Keeping that in mind, our focus is to compare the logical architecture of ROS2 and Adaptive AUTOSAR. Kampmann et al. [14] present a flexible SOA and analyze the characteristics in contrast to existing solutions. While they do show the similarities and differences of their architecture and ROS2 as well as Adaptive AUTOSAR, they do not explicitly compare the latter two.

In addition, literature evaluating automotive dynamic architectures and SOA concepts based on either ROS2 or Adaptive AUTOSAR can be found. Maruyama et al. [8] analyze

the capabilities of ROS2 for embedded systems without an automotive focus. In [15] the interoperability of ROS2 and Adaptive AUTOSAR is evaluated for a real-time system using Data Distribution Service (DDS). While they do show the characteristics of the two platforms, the focus lies on their compatibility rather than pointing out the individual suitability in an automotive context.

There are a lot of automotive architecture examples based on Adaptive AUTOSAR highlighting the performance thereof with regard to future use cases or SOA, e.g. [2], [16], [17]. As they omit ROS2 they can not be compared to this work. On the other hand, very few but some evaluations analyze ROS2 in an automotive context and examine its fit, for example [9], [10].

The existing literature points out that both Adaptive AUTOSAR and ROS2 can be used in an automotive context. A comparison and evaluation of the two platforms is still missing, which is the aim of this work. Since the satisfaction of future requirements in automotive architectures is still a challenge, this comparison can provide a guideline emphasizing the role of E/E architecture standards.

III. SERVICE-ORIENTED ARCHITECTURES AND IMPLICATIONS ON ARCHITECTURE PLATFORMS

Automotive E/E architectures as a system of systems comprises SW and HW components as well as mechanical parts. The classic distributed architecture is characterized by HW and SW being tightly coupled on numerous ECUs [2]. In contrast, SOAs enable application SW and executing HW to be independent of one another [18]. To realize this loosely coupled architecture, the Operating Systems (OSs) and middleware occupy a central position. The middleware layer used to be mainly responsible for communication between ECUs. Henceforward requirements of a middleware include the execution of abstract and virtualized communication within the whole vehicle architecture or even with cloud and backend infrastructure [19]. The communication of these system components may not be known during system development. Therefore, communication is dynamic in an interconnected system with changing subsystems, and links need to be established flexibly during runtime. The high amount of data arising within a connected vehicle and the SOA lead to the integration of new protocols within the E/E architecture [20].

The automotive reference architecture for dynamic communication, as specified in Adaptive AUTOSAR, divides functional applications and a Portable Operating System Interface (POSIX)-compliant OS with a modular layer called AUTOSAR Runtime Environment for Adaptive Applications (ARA) (see Fig. 2). Within the ARA, interfaces and services are defined to specify OS access and a communication middleware that enables the realization of a SOA based on Ethernet. The first release of Adaptive AUTOSAR stated Scalable service-Oriented MiddlewareE over IP (SOME/IP) as the middleware protocol for an accordingly standardized architecture [17]. Next to the automotive-specific middleware solution, DDS is a non-automotive-specific alternative. While

Adaptive AUTOSAR included DDS from release 18-10 onwards [21], ROS2 was built on DDS from the beginning, not offering any other middleware (see Fig. 3) [22].

The integration of DDS is an example that a non-industry specific middleware solution enriched the automotive standard Adaptive AUTOSAR.

Alongside this development, ROS2, originating from robotic applications, enters the automotive industry. In contrast to Adaptive AUTOSAR, ROS2 is open source and neither a membership nor license fees apply. The unadjusted implementation as provided by the ROS community is not officially certified for real-time applications of vehicle systems. But yet, a commercial Real-time Operating System (RTOS) developed by automotive SW companies and based on ROS2 recently got the certification for ASIL-D applications [10]. Motivated by these developments, the following comparison of the architecture of ROS2 (Fig. 3) based on Adaptive AUTOSARs modular structure (Fig. 2) is made.

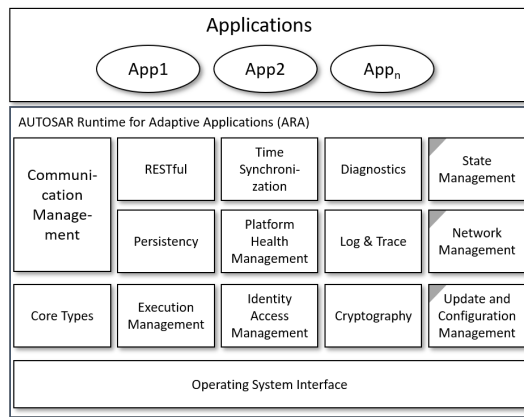


Fig. 2: Structure of Adaptive AUTOSAR

IV. COMPARISON OF ROS2 AND ADAPTIVE AUTOSAR

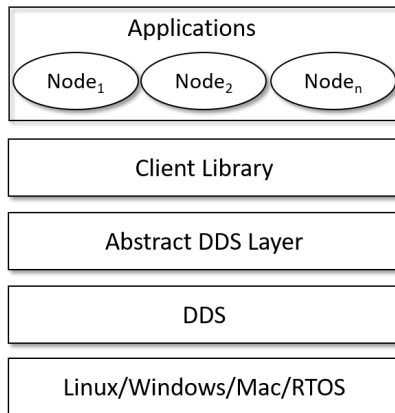


Fig. 3: Structure of ROS2

Adaptive AUTOSAR and ROS2 provide layers separating executing HW and applications (Fig. 2, Fig. 3). Comparing the overall structure of the platforms (Fig. 2, Fig. 3)

it is striking that the Adaptive AUTOSAR Application Programming Interface (API) names give an impression of their intended functionality. ROS2s layers are not self-explanatory to the same extent. Consequently, a comparison of the scope and content of ROS2s Client Library and Abstract DDS Layer and the Adaptive AUTOSAR architecture needs to be made. As the latter one is the automotive standard and we want to examine Robot Operating System 2s (ROS2s) suitability for the industry, we will compare the platforms on the basis of Adaptive AUTOSARs functionalities and core assets.

The core of the Adaptive AUTOSAR architecture is the ARA containing Adaptive APIs (see Fig. 2), provided by functional clusters. Functional clusters are classified as Adaptive Platform Foundation or Adaptive Platform Services, providing APIs for accessing ARA and services to applications within the platform. They describe the SW platform behavior from an application and network viewpoint but do not specify the SW design further. The Adaptive applications can offer services to one another or interact with non-platform services [23].

The following paragraphs introduce the Adaptive AUTOSAR architecture and the ROS2 correspondences, summarising their functionality concisely and emphasizing major differences. An overall comparison is provided in Table I.

1) Communication Management: In Adaptive AUTOSAR, the communication and information sharing of applications is organized by the Communication Management (CM) API package [4]. Service-oriented communication between all levels of the platform is realized and supervised thereby. CM does also provide intra-machine and inter-machine communication. The API specified SOME/IP to be the implemented middleware protocol, DDS was added with release 18-10 [2]. While DDS and SOME/IP do have common features, there are also key differences. SOME/IP realizes an object-based service-oriented communication. Service classes for subscribers need to be pre-defined during development, making it static to a certain degree [20]. In contrast, DDS enables dynamic runtime discovery by senders and receivers [24] and offers more interoperability and flexibility due to the non-necessity of static service specifications. DDS is based on Real-time Transport Protocol (RTPS) for publish-subscribe functionality, leveraging this wire protocol by providing advanced data-management features. In addition, further protocols can be added [25]. In SOME/IP, the information can be published to all subscribers broadcasting it using User Datagram Protocol (UDP) or Transmission Control Protocol (TCP) [26]. The event-based communication in SOME/IP can provide data of a current snapshot, field-based communication transmits data with read and write permission and data history for the receiver. Another feature is the method-based Remote Procedure Call (RPC) enabling a client to start a specific function on request with the purpose of receiving a message with data in return [20]. DDS defines three ways of communicating: messages can

be sent to subscribers depending on their topic or a service can be called by a client and will be delivered to the receiver. The third mode combines the approaches: a goal is published, triggering the feedback to be periodically sent to the receiver [27].

Communication paths as well as service definitions in Adaptive AUTOSAR can be established during development, at system start or during runtime dynamically. Besides, Adaptive AUTOSAR specifies signal-based network binding to allow the conversion of an accordingly communicating application with a service-oriented communicating application [28].

ROS2 defines a middleware API called *rmw* to enable the application-independent integration of DDS combined with RTPS as the only middleware framework solution. Both, Adaptive AUTOSAR and ROS2 offer inter-process communication being zero-copy capable and standardized in the automotive industry [29]. Besides, the communication in ROS2 is organized using nodes. Each node provides a function to the system. The publish-subscribe communication is realized in a decentralized way, where nodes discover each other without an intermediate master. In contrast to Adaptive AUTOSAR, nodes in ROS2 need to be integrated at design time, but service discovery happens exclusively during runtime. This is a key difference between the two platforms. The Adaptive AUTOSAR CM makes it possible to reduce overhead where dynamic communication is not required.

2) *Execution Management*: The Adaptive AUTOSAR Execution Management (EM) API realizes functionalities related to platform and application life cycle management including authentication, start and shut down of applications. EM interacts with the OS to schedule applications during runtime, thereby applications can be organized according to their priority [30]. The Execution Manifest specifies additional EM related topics. It specifically defines the possibility to statically set resources for specific applications as well as communication and thereby control resource allocation [5].

Execution management in ROS2 is organized by launch files within the *rcl* API. Execution is realized by Executors, a concept based on using threads of the OS to realize callbacks, execute timers, service servers and action servers on incoming messages and events [22]. While the standard executor of ROS2 cannot guarantee determinism, there are adjustments to be made to reach this goal. However, this adjustment can so far only be found in commercial solutions based on ROS2 and is not part of the standard open-source implementation [10]. Apart from that, further real-time and efficiency issues arise using ROS2 standard executors [31]. Therefore it seems that progress still has to be made towards satisfying automotive requirements in this context.

3) *Core Types*: The Core Type API package defines the basic requirements which apply to all functional clusters of the Adaptive platform. It contains a central

initialization function for all included shared libraries of the ARA layer, which is advised to be called an entry point for each Adaptive application. Furthermore, it describes error handling and therefore separates between an error, a violation or a corruption. An error is a return value of an Adaptive function which often is caused as a consequence of the input data. A violation refers to the internal state of the Adaptive platform and is not recoverable. Corruption is the consequence of a corrupt system resource such as the stack, heap or HW overflow. It is expected that for an application developer, only errors might occur since violations or corruptions would indicate a serious problem within the systems environment, such as faulty HW. Furthermore, the Core Type API defines commonly used data types within the Adaptive platform. While the platform type package describes standard types, such as integers, doubles or floats, the Core Type package defines data types such as arrays and vectors. Additionally, it defines return data types in the event of exceptions or errors [32].

Analogous to Adaptive AUTOSAR, the ROS2 client library *clcpp* needs to be initialized first (see Fig. 3). The client library also contains the definition of errors, such as invalid node or topic names, wrong function arguments or runtime errors. However, a clear separation between errors, violations or corruptions is not described. Thus, it can not be derived which error led to an abortion of the whole process or threw an exception.

ROS2 applications typically communicate through interfaces, being either a message, service or an action. The interfaces contain multiple fields with a field type and a field name. The build-in field types are booleans, integers, chars, floats, doubles and strings. They can be used to define static, bounded or unbounded dynamic arrays or a bounded string, which in combination can be used to create custom messages. This aspect can be evaluated as similar to Adaptive AUTOSAR.

4) *RESTful*: While the CM API package covers the service-oriented communication between Adaptive applications, the Representational State Transfer (RESTful) API package provides a web service-based communication option according to [33]. In contrast to RESTful services in the mobile world, which are only rarely exposed to resource constraints, it needed to be optimized for automotive applications to provide low and deterministic memory and processing time. An API in the CM Package is precisely defined down to the byte and therefore, mostly static data is processed. A RESTful API is typically characterized by dynamic data in combination with a Unified Resource Identifier (URI), processing its data asynchronously [34]. For the implementation of RESTful APIs, a dedicated ROS2 package could not be found. However, there is the ROSTful package available for ROS version Indigo or Kinetic. If and when this will be ported as ROS2 package can not be anticipated. An alternative way to realize asynchronous data processing in ROS2 are actions. They can be called by a client similar to a remote procedure call and thereby passing

over multiple data arguments. The action then processes the data and executes a callback once it's finalized. Even though an action is similar to an Adaptive AUTOSAR RESTful API, it is not the same, and ROS2 does not entirely fulfill the RESTful API characteristics.

5) *Persistency*: Adaptive AUTOSAR explicitly specifies the persistence of information beyond the boot and ignition in addition to the use of volatile storage [35].

ROS2, on the other hand, is limited to the underlying DDS implementation concerning data persistence. Cyclone DDS states that it will not use both transient and persistent memories until version 0.9. The current version 0.8.2 only supports volatile storage of data [36], a partial fulfillment of Adaptive AUTOSAR Persistency functionality can therefore be assumed.

6) *Time-Synchronisation*: Regarding time synchronization, both Adaptive AUTOSAR and ROS2 follow similar approaches. In Adaptive AUTOSAR, the master and slave time bases are synchronized. Network Time Protocol (NTP) is usually used for this, but Precision Time Protocol (PTP) is an alternative for reaching higher accuracy [37]. In ROS2, the messages can also be synchronized against each other and different clocks are available for that purpose. The same PTP is the solution achieving the highest accuracy [38].

7) *Platform Health Management*: Adaptive AUTOSAR monitors its applications with respect to timing constraints and logical program flows, as well as the state of the platform. When an error is detected, it notifies State Management, which determines error handling. Besides, Platform Health Management interacts with the HW watchdog and can trigger a watchdog response in the event of a critical fault and the notification to State Management would not be sufficient. However, the interfaces of the Health Management towards other functional clusters are only of informative nature and not standardized [39]. In ROS2, these functions are not implemented by default. Yet, following ROS2's open-source documentation, platform health management can be implemented. For example, the logging function, which can trigger various recovery routines in the event of errors, may be used to fulfill this functionality.

8) *Identity Access Management*: The Adaptive AUTOSAR Identity and Access Management (IAM) uses the x.509 certificate and is responsible for authentication and authorization of resource access. The IAM thus provides access control between ECUs and within an ECU [40]. ROS2 also works with permission files with the x.509 certificate within the provided DDS security plugin "DS:Access:Permission" to realize access control. The access control and the authorization mechanism of ROS2 use the public key infrastructure provided by the "DDS:Auth:PKI-DH" plugin of the DDS implementation [41]. The difference between the two platforms is mainly that AUTOSAR provides Adaptive Authentication and

Authorization also within an ECU, whereas ROS2 makes no statement in this respect. Available adjustments for this purpose could not be found.

9) *Diagnostics*: The Diagnostic API package describes a Unified Diagnostic Services (UDS) compliant implementation according to the corresponding ISO 14229-1 standard [42]. It currently provides functions for the Ethernet-based Diagnostic over IP (DoIP) layer but it is expected to be extended to include traditional automotive networks such as CAN [43].

ROS2 doesn't contain UDS compliant diagnostic functionalities. Even though the official ROS2 package index folder contains multiple diagnostic packages [22], they are not related to UDS and consequently not ISO 14229-1 compliant.

10) *Log and Trace*: Log and Trace in Adaptive AUTOSAR provides APIs to log and trace applications. The resulting information is categorized according to severity levels and can be linked to different cases, e.g., safety and security topics [44]. A standardized protocol by the AUTOSAR consortium is available for the representation of logging information as well as tools for the corresponding data analysis.

For ROS2, there are packages for logging and tracing available. Most of them are not part of the binary packages but can be integrated as needed. The tracing package enables the recording of runtime and execution data, system and component behavior, and analysis thereof. Analysis tools for the tracing files are also recommended [31]. A key disadvantage of ROS2 is the non real-time compliant logging functionalities as well as trustworthiness issues of certain log entries [10]. While it is not known to the authors if issues like these also arise in conjunction with Adaptive AUTOSAR, it is certainly a necessary requirement for safe and secure E/E architectures.

11) *Cryptography*: Concerning cryptographic procedures, the Adaptive AUTOSAR Platform defines its interface to the crypto stack "ara::sec::crypto" which includes random number generation, the Advanced Encryption Standard, asymmetric encryption Rivest-Shamir-Adleman (RSA), hash functions, Media-Access-Control-Address (MAC), and key exchange mechanisms. AUTOSAR also supports the handling of cryptographic objects, e.g., X.509 Certificates [45].

The security features of ROS2 are based on the security plug-in of the DDS standard. It is called "DDS:Crypto:AES-GCM-GMAC" and uses the Advanced Encryption Standard in the so-called Galois Counter Mode. The Public Key Infrastructure handles encryption, decryption, signing, and hashing. As all security features of DDS are disabled by default, the user must enable them via the ROS2 security tool [41]. Both Adaptive AUTOSAR and ROS2 offer the same functionalities, but for ROS2 manual activation is required.

The ARA also contains Platform Services (see Fig. 2), which will be described and compared to ROS2 equivalents in the following.

12) State Management: State Management receives events from other Adaptive AUTOSAR Platform Applications such as Platform Health Management, Diagnostic Management, Update and Configuration Management, and Network Management. The events include an event type, a priority and an application identifier as input for the State Manager to request state transitions from the execution manager. For instance, Platform Health Management can trigger an error recovery through the State Management service, which requests the execution manager to restart a corresponding function. Diagnostic Management can request to switch the system into different diagnostic states or initiate Update and Configuration Management to shut down system parts. For diagnostic and update purposes, Network Management will be used by the State Management to keep the network up and running during the diagnostic or update process [46]. ROS2 has no comparable diagnostic or update features. However, the recovery of a broken process can be realized using Quality-of-Service (QoS) characteristics. Thereby a process can be determined as dead in order to start recovery mechanisms. However, the QoS options are part of the underlying DDS implementation and only an indirect ROS2 feature.

13) Network Management: The Adaptive AUTOSAR Network Management acts detached from CM and manages the transition of operation, sleep mode and shutdown of network components. It follows a decentralized approach so that every node executes its activities independently according to Network Managements packets [47]. For ROS2 there is no specific network API for coordinating bus communication, but the QoS policies also specify the liveness of nodes according to communication transmitted and other communication related activities [22]. The Network Management of Adaptive AUTOSAR is an example for the very detailed specification of AUTOSAR APIs, while in ROS2 the functionality may be set up by combining several concepts and packages, but in this case a realization could not be found.

14) Update and Configuration Management: The Update and Configuration Management of Adaptive AUTOSAR is the interface handling update requests, the update itself and SW modification. It is responsible for realizing updates Over The Air (OTA) and combines the installation and update process with removal and track record of SW components for this purpose [47]. The service, which is similar to Linux packet managers (dpkg or Yellowdog Updater, Modified (YUM)) does only carry out authorized update SW. It also enables variant-specific configuration and validation of functionality and all other processes after the update [2] [48]. In ROS2, there are two packages for Continuous Integration (CI) and continuous development within the

buildfarm, the ROS2 scripts and templates provided by Open Robotics.

The Pull Request (PR) builds package runs and tests a package whenever a PR occurs. It is limited to Linux and to does not work across repositories [49]. To make up for these limitations, the CI builds package exists. In contrast to PR builds, it does not run and test automatically and its testing activity is not limited to the updated component but does test all packages on every platform. It is also possible to limit the test scope for time-related reasons. For updating functionality, the package maintainers must constantly do a package release. In total, the functionality seems to be comparable to the Adaptive AUTOSAR service.

V. SUMMARY

TABLE I: Overview of ROS2s fulfillment regarding Adaptive AUTOSAR API functionality

	Adaptive AUTOSAR API	ROS2 fulfillment level
Platform Foundation	Communication Management	-
	Execution Management	×
	Core Types	-
	RESTful	○
	Persistancy	-
	Time-Synchronisation	✓
	Platform Health Management	○
	Identity Access Management	-
	Diagnostics	×
	Log and Trace	○
Platform Services	Cryptography	✓
	State Management	-
	Network Management	×
	Update and Configuration Management	✓

✓ Complete fulfillment ○ Fulfillment with adjustments
 - Partial fulfillment (adjustment possibility not known) × No fulfillment (standard ROS2)

Table I illustrates the summary of the capabilities of ROS2 compared to the Adaptive AUTOSAR API functionality. However, in this context, it is worth mentioning that ROS and ROS2 were initially developed as a solution for robotic applications. From this starting point, it is now also used within existing autonomous vehicle solutions [50]. One reason for the success of ROS/ROS2 could be that automotive engineers may be looking for alternatives to AUTOSAR since the extent of the platform's standardized contents is enormous. The effort required to comply with the specification seems high, especially for developers with little AUTOSAR experience. Besides, the effort needed to comply with the standard is disproportionally increasing in relation to the size of a SW project [51]. ROS/ROS2 solutions seem to be promising alternatives. They are explicitly designed to realize interacting functions using nodes. The loosely coupled nature of these nodes leads to modularity, scalability, adjustability and enables faster development [52]. Additionally, ROS2 contains multiple features simplifying application development tasks.

One example are pre-defined, robotic-specific message types which can be used off-the-shelf to realize intra-node communication. Furthermore, ROS2 provides powerful tools for debugging code, recording, replaying and visualizing data. For example, RQT-Graph illustrates the communication between the nodes, thereby deriving the system's logical architecture bottom-up. Another example is RVIZ, which visualizes the sensor data and the vehicle itself in its environment.

Examining the way of development of the platforms, they are both following a model-based approach. For coding, the Adaptive AUTOSAR platform defines guidelines for using C++14 as a standard to enable more straightforward structuring of distributed systems [53]. The coding guidelines are based on existing guidelines in the automotive industry. The client libraries of ROS2 are code sets to ensure API usage conformity and applicability. These libraries enable users access to ROS2 concepts for building applications. There are client libraries in various programming languages. The ROS2 documentation also provides style guidelines for coding to support the goal of a consistent product.

One of the main differences between the platforms is that the standards of AUTOSAR are not open-source. A membership in the global consortium, comprising almost 300 organizations, which also contribute to the standards specification and evaluation, is necessary to access the documents. License fees apply and also AUTOSAR-licensed tools are not available free of charge.

In contrast, the ROS libraries and related documentation are open-source and the developer community provides additional packages and code bases that can be individually modified [52]. This is of significant importance since another limitation of AUTOSAR is the poor documentation [51]. [10] summarizes the here mentioned advantages as "...awesome development experience" by using ROS2.

Then again, there are ROS2-based solutions available that are not open source. Especially a recently certified and automotive-specific solution [10] is not publicly available. In this case, one may not take full advantage of the open source-related benefits, but the automotive-specific adaption seems beneficial.

If the most relevant gaps identified in Table I will be closed by the ROS community, the here mentioned benefits could let ROS expand from the autonomous driving domain to other automotive domains, thereby forming a realistic alternative to the well-established AUTOSAR standard.

VI. CONCLUSION

We have presented the architecture and content of Adaptive AUTOSAR and ROS2 in the context of prevailing automotive architecture development challenges. While the license-based Adaptive Platform is specifically tailored to the trend toward SOA in automotive, the initial scope of the open-source ROS2 were robot applications. Coming from the layers and APIs specified in Adaptive AUTOSAR, we explained the extent to which the ROS2 architecture corresponds to the Adaptive Functional Clusters of the AUTOSAR platform (as concluded in Table I).

Adaptive AUTOSAR is more standardized than ROS2, also providing development process standards. It serves interoperability and consistent specifications within the industry. The platform satisfies automotive requirements, especially regarding security innately.

While ROS2 does not specify as many services, APIs or process standards, packages for additional functionality are provided by the community. As Table I concludes, some of the Adaptive AUTOSAR APIs and service functionality can be completely fulfilled by ROS2. While the standard implementation does not provide all of the automotive related functionality as-is, adjustments can be made by adding packages and customize settings. On the other hand, there are characteristics of Adaptive AUTOSAR that ROS2 partly fulfills, but the discrepancy is not known to be fulfilled by additional packages. Then again, according to the analysis, there are APIs provided by Adaptive AUTOSAR where ROS2s fulfillment cannot be stated. As described in the individual sections of the APIs, these gaps may sometimes be obligatory (e.g. Network Management). In other cases, these differences are of more relevance regarding automotive requirements (e.g. Diagnostics).

The safety certification of a ROS2 based commercial version showed that it can be made automotive SOA-compliant by adapting and designing it accordingly. This explicitly refers to the enhancement of real-time capabilities and memory management by rewriting parts of the open-source implementation [10]. In contrast to Adaptive AUTOSAR, it may be barrier-free for new adopters to start using the platform in architecture development.

In future work, we want to get deeper insights regarding ROS2s industry readiness and automotive applicability. We also hope to obtain empirical evidence of the benefits and drawbacks of the platforms and future E/E-architectures in general. Specifically, we want to address the question of how innovation and standardization interfere with each other regarding contemporary architecture trends and requirements.

REFERENCES

- [1] H. Zhu, W. Zhou, Z. Li, L. Li, and T. Huang, "Requirements-driven automotive electrical/electronic architecture: A survey and prospective trends," *IEEE Access*, vol. 9, pp. 100 096–100 112, 2021.
- [2] A. Vetter, P. Obergfell, H. Guissouma, D. Grimm, M. Rumez, and E. Sax, "Development processes in automotive service-oriented architectures," in *2020 9th Mediterranean Conference on Embedded Computing (MECO)*. IEEE, 2020, pp. 1–7.
- [3] S. Frohn and F. Rees, "From Signal to Service - Challenges for the Development of AUTOSAR Adaptive Applications." Vector Informatik GmbH, 2018.
- [4] A. Arestova, M. Martin, K.-S. Hielscher, and R. German, "A Service-Oriented Real-Time Communication Scheme for AUTOSAR Adaptive using OPC UA and Time-Sensitive Networking," *Sensors*, vol. 21, p. 2337, 03 2021.
- [5] A. Bhat, S. Samii, and R. R. Rajkumar, "Fault-Tolerance Support for Adaptive AUTOSAR Platforms using SOME/IP," in *2020 IEEE 26th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*. IEEE, 2020, pp. 1–6.
- [6] M. Schindewolf, H. Guissouma, and E. Sax, "Analysis and modeling of Future Electric/Electronic Architectures for Modular Vehicles Concepts," in *21. Internationales Stuttgarter Symposium*, M. Bargende, H.-C. Reuss, and A. Wagner, Eds. Wiesbaden: Springer Fachmedien Wiesbaden, 2021, pp. 32–46.

- [7] S. Kugele, P. Obergfell, M. Broy, O. Creighton, M. Traub, and W. Hopfensitz, "On service-orientation for automotive software," in *2017 IEEE International Conference on Software Architecture (ICSA)*. IEEE, 2017, pp. 193–202.
- [8] Y. Maruyama, S. Kato, and T. Azumi, "Exploring the performance of ROS2," in *Proceedings of the 13th International Conference on Embedded Software*. ACM, Oct. 2016.
- [9] M. Reke, D. Peter, J. Schulte-Tigges, S. Schiffer, A. Ferrein, T. Walter, and D. Matheis, "A self-driving car architecture in ROS2," in *2020 International SAUPEC/RobMech/PRASA Conference*. IEEE, 2020, pp. 1–6.
- [10] J. Becker, M. Sagar, and D. Pangercic, "A safety-certified vehicle OS to enable software-defined vehicles," in *Proceedings*. Springer Fachmedien Wiesbaden, 2021, pp. 51–67.
- [11] C. Berger and M. Dukaczewski, "Comparison of architectural design decisions for resource-constrained self-driving cars - a multiple case-study," *Informatik 2014*, 2014.
- [12] D. Trachtenherz, "Logische Architektur," in *Eigenschaftsorientierte Beschreibung der logischen Architektur eingebetteter Systeme*. Vieweg+Teubner, 2010, pp. 19–27. [Online]. Available: https://doi.org/10.1007/978-3-8348-9703-9_2
- [13] C. Baun, *Computer Networks / Computernetze*. Springer Fachmedien Wiesbaden, 2019. [Online]. Available: <https://doi.org/10.1007/978-3-658-26356-0>
- [14] A. Kampmann, B. Alrifae, M. Kohout, A. Wüstenberg, T. Woopen, M. Nolte, L. Eckstein, and S. Kowalewski, "A dynamic service-oriented software architecture for highly automated vehicles," in *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*. IEEE, 2019, pp. 2101–2108.
- [15] N. Parmar, V. Ranga, and B. Simhachalam Naidu, "Syntactic Interoperability in Real-Time Systems, ROS 2, and Adaptive AUTOSAR Using Data Distribution Services: An Approach," in *Inventive Communication and Computational Technologies*. Springer, 2020, pp. 257–274.
- [16] G. Reichart and R. Asmus, "Progress on the AUTOSAR Adaptive Platform for Intelligent Vehicles," in *Automatisiertes Fahren 2020*. Springer, 2021, pp. 67–75.
- [17] S. Fürst and M. Bechter, "AUTOSAR for connected and autonomous vehicles: The AUTOSAR Adaptive Platform," in *2016 46th annual IEEE/IFIP international conference on Dependable Systems and Networks Workshop (DSN-W)*. IEEE, 2016, pp. 215–217.
- [18] D. Zerfowski and A. Lock, "Functional architecture and E/E-Architecture - A challenge for the automotive industry," in *19. Internationales Stuttgarter Symposium*. Springer, 2019, pp. 909–920.
- [19] O. Burkack, J. Deichmann, G. Doll, and C. Knochenhauer, "Rethinking car software and electronics architecture," *McKinsey & Company*, 2018.
- [20] M. Rumez, D. Grimm, R. Kriesten, and E. Sax, "An overview of automotive service-oriented architectures and implications for security countermeasures," *IEEE access*, vol. 8, pp. 221 852–221 870, 2020.
- [21] "Adaptive AUTOSAR - Specification of Communication Management," AUTOSAR, Standard, Oct. 2018.
- [22] "ROS2 Documentation," <http://docs.ros.org>, accessed: 2022-01-10.
- [23] "AUTOSAR," <http://www.autosar.org>, accessed: 2022-01-11.
- [24] H. Stoll, D. Grimm, M. Schindewolf, M. Brodatzki, and E. Sax, "Dynamic Reconfiguration of Automotive Architectures Using a Novel Plug-and-Play Approach," in *2021 IEEE Intelligent Vehicles Symposium Workshops (IV Workshops)*. IEEE, 2021, pp. 70–75.
- [25] A. Corsaro and D. C. Schmidt, "The data distribution service: The communication middleware fabric for scalable and extensible systems-of-systems," *System of Systems*, vol. 19, 2012.
- [26] "Adaptive AUTOSAR - SOME/IP Protocol Specification," AUTOSAR, Standard, Nov. 2016.
- [27] Y. Liu, Y. Guan, X. Li, R. Wang, and J. Zhang, "Formal analysis and verification of DDS in ROS2," in *2018 16th ACM/IEEE International Conference on Formal Methods and Models for System Design (MEMOCODE)*, 2018, pp. 1–5.
- [28] "Adaptive AUTOSAR - Specification of Communication Management," AUTOSAR, Standard, Nov. 2019.
- [29] "Eclipse foundation," {<https://newsroom.eclipse.org/eclipse-newsletter/2021/june/qa-it%E2%80%99s-time-take-closer-look-eclipse-iceoryx>}, accessed: 2022-03-04.
- [30] "Adaptive AUTOSAR - Specification of Execution Management," AUTOSAR, Standard, Nov. 2019.
- [31] C. Bédard, I. Lütkebohle, and M. Dagenais, "ros2_tracing: Multipurpose Low-Overhead Framework for Real-Time Tracing of ROS 2," *arXiv preprint arXiv:2201.00393*, 2022.
- [32] "Adaptive AUTOSAR - Specification of Core Types," https://www.autosar.org/fileadmin/user_upload/standards/adaptive/21-11/AUTOSAR_SWS_AdaptivePlatformCore.pdf, accessed: 2022-03-01.
- [33] R. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, 01 2000.
- [34] "Adaptive AUTOSAR - Specification of RESTful," https://www.autosar.org/fileadmin/user_upload/standards/adaptive/21-11/AUTOSAR_SWS_RESTfulCommunication.pdf, accessed: 2022-03-01.
- [35] "Adaptive AUTOSAR - Specification of Persistancy," Nov. 2021.
- [36] "Cyclone DDS Roadmap," <https://github.com/eclipse-cyclonedds/cyclonedds/blob/master/ROADMAP.md>, accessed: 2022-02-09.
- [37] "Adaptive AUTOSAR - Specification of Time Synchronization," AUTOSAR, Standard, Nov. 2021.
- [38] C. S. V. Gutiérrez, L. U. S. Juan, I. Z. Ugarte, I. M. Goenaga, L. A. Kirschgens, and V. M. Vilches, "Time synchronization in modular collaborative robots," *arXiv preprint arXiv:1809.07295*, 2018.
- [39] "Specification of Platform Health Management for Adaptive Platform," AUTOSAR, Standard, Nov. 2020.
- [40] "Adaptive AUTOSAR - Specification of Identity and Access Management," Mar. 2018.
- [41] "ROS 2 DDS-Security integration," https://design.ros2.org/articles/ros2_dds_security.html, accessed: 2022-02-01.
- [42] I. S. Organization, *ISO 14229-1: Road Vehicles - Unified Diagnostic Services (UDS) - Part 1: Specification and Requirements*. ISO, 2004. [Online]. Available: <https://books.google.de/books?id=GNIDtwAACAAJ>
- [43] "Adaptive AUTOSAR - Specification of Diagnostic," https://www.autosar.org/fileadmin/user_upload/standards/adaptive/21-11/AUTOSAR_SWS_Diagnostics.pdf, accessed: 2022-03-01.
- [44] M. Staron, *Automotive software architectures*. Springer, 2021.
- [45] "Adaptive AUTOSAR - Specification of Cryptography," Nov. 2021.
- [46] "Adaptive AUTOSAR - Specification of State Management," https://www.autosar.org/fileadmin/user_upload/standards/adaptive/21-11/AUTOSAR_SWS_StateManagement.pdf, accessed: 2022-03-01.
- [47] "Adaptive Platform Release Overview," AUTOSAR, Standard, Nov. 2021.
- [48] "Adaptive Platform - Specification of Update and Configuration Management," AUTOSAR, Standard, Nov. 2021.
- [49] "ROS 2 Package Maintainer Guide," <https://docs.ros.org/en/foxy/How-To-Guides/Package-maintainer-guide.html#continuous-integration>, accessed: 2022-02-10.
- [50] W. Shi and L. Liu, *Computing Systems for Autonomous Driving*. Springer International Publishing, 2021. [Online]. Available: https://books.google.de/books?id=sh1_zgEACAAJ
- [51] S. Martínez-Fernández, C. P. Ayala, X. Franch, and E. Y. Nakagawa, "A survey on the benefits and drawbacks of autosar," in *Proceedings of the First International Workshop on Automotive Software Architecture*, 2015, pp. 19–26.
- [52] A. Gavrilov, M. Bergaliyev, S. Tinyakov, and K. Krinkin, "Analysis of robotic platforms: Data transfer performance evaluation," in *Conference of Open Innovations Association, FRUCT*, no. 29. FRUCT Oy, 2021, pp. 437–443.
- [53] M. Zielinski, "Aufschlüsselung der AUTOSAR C ++ 14-Codierungsrichtlinien für adaptives AUTOSAR," <http://de.parasoft.com/blog/breaking-down-the-autosar-c14-coding-guidelines-for-adaptive-autosar/>, accessed: 2022-01-11.